
title: PROJET IOC

PROJET IOC

Auteurs :

Zahir Sami Amoura

Taha Sebai

Important links :

<https://randomnerdtutorials.com/raspberry-pi-publishing-mqtt-messages-to-esp8266/>

Journal De bord :

- au debut on va juste essayer d'implementer un simple broqueur mqtt sur la raspberrypi qui communique avec la esp32

Setting up the Raspberry

- alors c'etait un peu une galere (reseau n'est pas mon point fort mais ça va changer hehe) au debut je voulais creer a static ip avec mon partage de connexion hotspot ce qui y a bien marché au debut mais apres chaque connexion le mask change haha du coup mon pc plus acceder à la raspberry puisque ils ont pas le meme mask vu que la raspberry elle a une adresse static avec un mask fix
- pour résoudre ça j'ai juste viré l'adresse static et j'ai commencé à me connecter utilisant le prefix .local ce qui y est bien plus pratique, puis je l'ai enregistré sur mon ssh-config comme sakina maintenant je peux y accéder avec ssh sakina.
Ps : j'ai changé le mdp aussi (completely took it off)

Setting up Mosquitto MQTT broker

- followed this tuto : <https://randomnerdtutorials.com/how-to-install-mosquitto-broker-on-raspberry-pi/>

Implémentation de MQTT sur les ESP32

Une fois la partie Raspberry Pi terminée, nous nous sommes attaqués sur l'implémentation des ESP32 (création d'un client qui est abonné et client).

Pour le moment, on se concentre sur la preuve de concept. Dans notre cas, nous voulons que lorsqu'on veut allumer la LED de la carte depuis le site, celle-ci s'allume. Cette expérience permettra de confirmer le bon fonctionnement du serveur ainsi que la communication entre les deux cartes.

Pour la communication de l'ESP32 à travers MQTT, nous allons utiliser la librairie **PubSubClient** de Nick O'Leary. Il existe de nombreuses autres librairies (parfois plus efficaces) pour répondre à notre besoin, mais notre choix s'est porté sur celle-ci car elle est bien documentée, et relativement facile d'utilisation.

Rapport

L'objectif de notre projet est de créer un petit jeu, sans véritable logique derrière, pour simplement démontrer le bon fonctionnement de notre projet. L'idée est que les deux cartes envoient des "attaques" que l'on choisit directement depuis le site web, et qu'elles puissent lancer ces dernières lorsqu'on appuie sur le bouton poussoir. Les attaques et défenses seront visibles à travers des animations sur l'écran OLED.

Conceptuellement, on communique avec les ESP32 depuis le site web (pour choisir les attaques), puis ces dernières communiquent entre elles pour envoyer les attaques.

La dernière étape est de mettre en place le serveur web pour interagir avec les cartes à travers une page HTML, ainsi que le broker MQTT qui assure la communication entre les différents modules. Pour cela, nous nous sommes aidés d'un tutoriel trouvé sur Internet : <https://randomnerdtutorials.com/raspberry-pi-publishing-mqtt-messages-to-esp8266/>

Pour le broker, nous utiliserons **mosquitto** (on le lancera en arrière plan sur la raspberry pi). Cette librairie nous a été présentée par monsieur Wajsburt, nous l'avons déjà utilisée lors du tp5. C'est la raison pour laquelle nous l'avons choisie. Pour le serveur web, nous avons décidé de l'implémenter en utilisant le framework **Flask**. Elle est utilisée par le tutoriel qui nous a inspirés, et elle permet aussi de faciliter l'implémentation du serveur et le routage des applications.

On utilise aussi la librairie **paho-MQTT** pour que l'application web puisse communiquer avec les ESP32. On crée un client sur le serveur pour qu'il puisse lui aussi s'abonner à des topics et publier.

Pour mener à bien ce projet, nous avons commencé par la preuve de concept. Dans notre cas, nous voulons que lorsqu'on veut allumer la LED de la carte depuis le site, celle-ci

s'allume. Cette expérience permettra de confirmer le bon fonctionnement du serveur ainsi que la communication entre les deux cartes.

Interface Web :

L'interface web sert principalement à choisir le type d'attaque que nous voulons lancer sur les esp32.



Interface graphique

Lorsque vous cliquez sur un bouton (Freeze ou Lightning), l'application web envoie une requête au script Python.

Le script Python traduit cette action en un message binaire (1 ou 0) et le publie immédiatement sur un sous-topic spécifique (ESP32/freeze ou ESP32/zap).

Code ESP32 :

L'Architecture Globale

Le système est configuré comme un jeu ou un dispositif d'effets interactifs en réseau :

- Le Serveur Central (Raspberry Pi) : Il héberge le broker MQTT. Il reçoit les messages en provenance des ESP32 et du serveur Web, ensuite les distribue aux appareils ESP32.
- Les Terminaux (Les deux ESP32) : Ils exécutent le même code (à part les topics sont différents). Chaque carte gère de manière autonome un écran OLED, un bouton physique connecté sur la broche 23, et communique en temps réel via le réseau Wi-Fi.

Fonctionnement du Code sur l'ESP32

1. Gestion du Bouton Physique (Attaque sortante)

Lors d'une pression sur le bouton de l'ESP32 :

- Le programme vérifie la valeur de la variable `current` pour identifier l'attaque sélectionnée (0 pour Freeze, 1 pour Lightning).
- L'écran OLED local affiche une animation de projectile en mouvement (`activeAttack->drawProjectile()`).

Une fois l'animation terminée (`ATTACK_DONE`), l'ESP32 publie un message MQTT vers l'autre ESP32 (`Attack_freeze_1` ou `Attack_zap_1`) le numéro désigne la cible de l'attaque

pour notifier l'envoi de l'attaque.

2. Réception MQTT (Sélection et Impacts subis)

La fonction `callback()` intercepte en continu les messages diffusés par la Raspberry Pi (envoyés par l'autre ESP32 ou par l'interface web) :

- Changement de mode (ESP32/freeze) : Si l'interface web transmet une instruction sur ce canal, l'ESP32 met à jour sa variable `current`. Cela modifie le type de sort qui sera généré lors du prochain appui sur le bouton.
- Réception d'un impact (`Attack_freeze_2` ou `Attack_zap_2`) le numero designe l'ESP attaquant : Si l'autre module valide une attaque, l'ESP32 reçoit l'information et déclenche immédiatement l'animation d'impact correspondante sur son propre écran OLED (`Attacked->drawImpact()`).

Nous discuterons les animations dans la prochaine section.

Pour recapituler l'interface web designe le type de sort lance dans chaque ESP32 en publiant dans un sous topic (zap et freeze dans le code ils designent qu'elle carte est vise zap pour la premiere et freeze pour la deuxieme, ce n'est pas intuitif mais j'ai pas eu le temps de changer du coup 0 c'est pour le freeze et 1 pour le lighting). Puis a l'appui du bouton on lance le sort on appercoie un petit projectile sur l'ecran oled et une fois arriver au bout il envoie un signal via mqtt pour prevenir l'autre esp32 qu'il est entrain d'etre attaquer, on vera sur l'autre esp32 une animation d'impact.

Animation :

Pour les animations nous avons utilise les concepts de la programmation orientée objet en C++ pour orchestrer l'affichage de deux attaques distinctes (Freeze et Lightning/Zap) sur un écran OLED SSD1306. L'utilisation d'une classe héritée permet d'exécuter la logique physique globale tout en personnalisant l'aspect visuel de chaque sort.
pour ces classes nous allons utilise aussi les bibliotheques generique de l'ecran oled.

La Classe de Base : AttackAnimation

La classe `AttackAnimation` fait office de moteur logique et temporel commun. Elle gère les variables de position, de vitesse et les transitions de cycles, sans imposer de dessin spécifique pour les projectiles ou les impacts.

Machine à états (`_state`)

Elle encadre la durée de vie d'une action à travers quatre étapes de l'énumération `AttackState` :

- **ATTACK_READY** : L'animation est initialisée et en attente d'activation.
- **ATTACK_MOVING** : Le projectile est actif et progresse horizontalement sur l'écran.
- **ATTACK_IMPACTING** : Le projectile a atteint sa cible géographique, l'effet visuel de collision se déploie.
- **ATTACK_DONE** : La séquence d'impact est entièrement écoulée, signalant la fin de l'animation.

Le Moteur Physique (update)

À chaque itération, la méthode `update()` incrémente la coordonnée horizontale `_xPos` en fonction du paramètre de vitesse `_speed`. Dès que la coordonnée franchit la bordure droite de la dalle (`_display.width()`), l'état bascule sur **ATTACK_IMPACTING** et enclenche un compte à rebours basé sur la variable `_impactTimer` qui gere le temps de l'animations d'impact.

La Boîte de Texte (`drawTextBox`)

Il s'agit d'une méthode utilitaire partagée par toutes les sous-classes. Elle calcule les dimensions adaptées à la longueur de la chaîne de caractères `_name` et dessine un rectangle noir opaque ceinturé de blanc au centre de l'écran, facilitant la lecture du texte de l'attaque. Elle contient les noms d'effets des sorts (zap and freeze).

Les Sous-Classes Visuelles

Chaque sous-classe hérite du gestionnaire d'états de la classe de base, mais redéfinit le comportement des fonctions graphiques grâce au mécanisme de polymorphisme (`override`).

Classe `FreezeAttack`

Cette classe est spécialisée dans la modélisation graphique d'un sort de glace.

- **Projectile** (`drawProjectile`) : Génère un tracé géométrique simple sous la forme d'un cercle blanc évidé en mouvement.
- **Impact** (`drawImpact`) : Simule un effet de givre en illuminant simultanément 60 pixels unitaires (`drawPixel`) positionnés de façon aléatoire sur la totalité de la surface de l'écran, avant de superposer l'encadré textuel "FREEZE".

Classe `LightAttack`

Cette classe est spécialisée dans la modélisation graphique d'un sort électrique (Zap).

- Projectile (`drawProjectile`) : Génère un repère visuel composé d'un triangle plein pointant vers l'avant, simulant une flèche d'énergie.

Impact (`drawImpact`) : Combine deux techniques d'animation dynamique :

- Particules électriques : Génère 40 segments verticaux de longueurs et de coordonnées aléatoires à chaque rafraîchissement d'image pour traduire le scintillement d'un arc de tension.
- Effet stroboscopique (Flash) : Introduit une probabilité d'apparition (1 chance sur 5 par cycle) pour saturer l'écran de lignes blanches verticales continues, émulant l'aveuglement produit par la foudre. La boîte de texte "ZAP" vient finaliser l'affichage.

Intégration dans le Projet Global

Cette architecture est optimale pour l'ESP32 car elle implémente des fonctions virtuelles pures (`virtual ... = 0`). Le programme principal manipule uniquement un pointeur générique de type `AttackAnimation*`.

vu que l'impact dure plusieurs tick, il faut a chaque fiin de frame faire un `display.display()` et un `clearDisplay` apres pour avoir l'effet de l'animations.

Qu'il s'agisse de la structure de tir ou de la routine d'impact, l'appel des méthodes de dessin s'adapte automatiquement à la nature de l'objet instancié (glace ou foudre) choisi en amont par l'utilisateur sur l'interface web.

Guide d'Implémentation du Projet

Ce guide détaille les étapes nécessaires pour déployer et interconnecter les différents composants du projet : le broker MQTT, les modules ESP32, les bibliothèques d'animation et l'interface utilisateur Flask.

1. Configuration du Broker MQTT (Mosquitto)

Le broker MQTT est installé sur la Raspberry Pi et sert de commutateur central pour les messages. Pour faciliter le développement, la configuration choisie permet un accès distant sans authentification.

Installation

Exécutez la commande suivante pour installer le serveur Mosquitto ainsi que ses outils en ligne de commande :

```
sudo apt install -y mosquitto mosquitto-clients
```

Activation du service

Pour s'assurer que Mosquitto se lance automatiquement à chaque démarrage de la Raspberry Pi, activez son service système :

```
sudo systemctl enable mosquitto.service
```

Configuration des accès distants

Par défaut, Mosquitto bloque les connexions extérieures. Pour autoriser les modules ESP32 et le serveur Web à communiquer librement, ouvrez le fichier de configuration :

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Naviguez jusqu'à la fin du fichier et ajoutez les deux lignes suivantes :

```
listener 1883  
allow_anonymous true
```

Redémarrage du broker

Enregistrez les modifications (Ctrl+O, puis Entrée) et quittez l'éditeur (Ctrl+X). Appliquez les changements en redémarrant le service :

```
sudo systemctl restart mosquitto
```

2. Connexion Réseau (ESP32 et Raspberry Pi)

Pour que tous les composants s'échangent des informations, la Raspberry Pi et les deux cartes ESP32 doivent être connectées au même réseau local, via le partage de connexion d'un smartphone.

Dans le fichier source .ino de nos programmes ESP32, mettez à jour les variables de configuration réseau avec vos identifiants :


```
const char* ssid = "Votre_Partage";  
const char* password = "Mot_De_Passe_Du_Partage";  
const char* mqtt_server = "Adresse_IP_De_La_Raspberry_Pi";
```

Note : L'adresse IP de la Raspberry Pi peut être récupérée sur son terminal via la commande

```
hostname -I
```

3. Intégration des Animations OLED

Les visuels des attaques reposent sur des classes spécifiques. Pour que le code principal de l'ESP32 puisse compiler et utiliser ces animations :

- Localisez le répertoire nommé Attack contenant nos fichiers d'animation.
- Copiez et déplacez ce dossier complet dans le dossier libraries de votre environnement de développement Arduino (généralement situé dans Documents/Arduino/libraries/).
- Une fois le dossier copié, les fonctionnalités de la bibliothèque seront accessibles.

4. Lancement de l'Interface Web Flask

L'interface de contrôle graphique est sur la Raspberry Pi.

Déploiement des fichiers

Copiez le script Python app.py et le dossier contenant les pages HTML templates/ dans le répertoire de travail de votre Raspberry Pi.

Démarrage du serveur

Exécutez le script Python pour instancier le serveur Web :

```
python3 app.py
```

Accès au tableau de bord

Ouvrez le navigateur internet d'un appareil (PC ou smartphone) également connecté au partage de connexion, puis saisissez l'adresse de la Raspberry Pi suivie du port 8181 :

`http://<ADRESSE_IP_RPI>:8181`

5. Procédure de Démarrage Global

Pour initialiser l'ensemble du système :

- Assurez-vous que le partage de connexion est actif et que la Raspberry Pi y est connectée.
- Lancez le serveur Flask sur la Raspberry Pi.
- Alimentez et téléversez les scripts sur les deux cartes ESP32.
- Vérifiez via le moniteur série de l'IDE Arduino que les cartes réussissent leur connexion au point d'accès Wi-Fi, puis qu'elles se synchronisent correctement au broker MQTT.
- Utilisez l'interface web pour attribuer les sorts et appuyez sur les boutons physiques pour déclencher les séquences d'attaque.